

Ejercicio resuelto paso a paso

Asignación de memoria: primer ajuste, mejor ajuste y peor ajuste

Bloques libres no contiguos: 100K, 500K, 200K, 300K y 600K

Enunciado

Bloques libres *no continuos* (orden físico y de lista): 100K, 500K, 200K, 300K, 600K. Operaciones en orden: **(1)** asignar A = 212K, **(2)** asignar B = 417K, **(3)** asignar C = 112K, **(4)** liberar A, **(5)** asignar D = 426K, **(6)** compactación. Condiciones: bloque mínimo de 1K, máximo 10K de fragmentación interna, sin apuntador vagabundo, sin compactación por demanda.

Interpretación de las reglas del enunciado

- **No continuos:** los 5 bloques son islas separadas; al liberar, solo hay coalescencia con huecos vecinos *dentro de la misma isla*.
- **Sin apuntador vagabundo:** primer ajuste escanea siempre *desde el inicio* de la lista (el vagabundo sería la variante *next fit*).
- **Frag. interna máx. 10K:** si el sobrante fuera $\leq 10K$ se entregaría el hueco completo. *En este ejercicio nunca se activa:* el sobrante mínimo que aparece es 83K, así que todos los huecos se parten (asignación exacta + sobrante libre).
- **Sin compactación por demanda:** si una solicitud no cabe, **falla**; solo se compacta en el paso 6, cuando se pide explícitamente.

Idea clave

Primer ajuste: el primer hueco (desde el inicio) donde quepa. **Mejor ajuste:** el hueco más pequeño donde quepa (menor sobrante). **Peor ajuste:** el hueco más grande disponible. En los diagramas: gris = libre (el número es su tamaño en K), colores = bloques asignados, espacios en blanco = fronteras entre islas no contiguas. Todo a escala.

1. Primer ajuste (First Fit)

Regla: recorrer la lista de huecos desde el inicio y tomar el primero donde quepa la solicitud.

Estado inicial. La lista de huecos, en orden físico: 100, 500, 200, 300, 600 (total libre: 1700K).



Paso 1 — Asignar A = 212K. Escaneo desde el inicio: ¿100 ≥ 212? no. ¿500 ≥ 212? **sí** → se asigna ahí. Sobrante: 500 − 212 = 288K > 10K, así que el hueco se parte. Huecos: 100, 288, 200, 300, 600.



Paso 2 — Asignar B = 417K. Escaneo: 100 no, 288 no, 200 no, 300 no, **600 sí**. Sobrante: 600 − 417 = 183K. Huecos: 100, 288, 200, 300, 183.



Paso 3 — Asignar C = 112K. Escaneo: 100 no, **288 sí** (primer hueco que alcanza; está dentro de la vieja isla de 500). Sobrante: 288 − 112 = 176K. Huecos: 100, 176, 200, 300, 183.



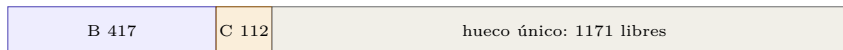
Paso 4 — Liberar A. El bloque de 212K vuelve a estar libre. ¿Coalescencia? Su vecino derecho es C (asignado) y a su izquierda está la frontera de la isla ⇒ *no se fusiona con nadie*. Huecos: 100, 212, 176, 200, 300, 183 (total: **1171K**).



Paso 5 — Asignar D = 426K. Escaneo completo: ningún hueco alcanza (el mayor es 300K). Sin compactación por demanda ⇒ la solicitud **FALLA** y el estado de la memoria no cambia. Esto es *fragmentación externa*: hay 1171K libres en total, pero dispersos.

FALLA: 1171K libres pero el hueco mayor es de 300K (fragmentación externa)

Paso 6 — Compactación. Se mueven B y C a un extremo y todos los huecos se funden en **uno solo de 1171K**. Comprobación: 1700 − (417 + 112) = 1171 ✓. (Recién ahora D podría atenderse.)



2. Mejor ajuste (Best Fit)

Regla: revisar TODOS los huecos y elegir el más pequeño donde quepa (el de menor sobrante), para proteger los huecos grandes.

Estado inicial. La misma lista de huecos: 100, 500, 200, 300, 600.

100	500	200	300	600
-----	-----	-----	-----	-----

Paso 1 — Asignar A = 212K. Candidatos que alcanzan: 500, 300, 600, con sobrantes 288, **88** y 388. Gana el **300** (menor sobrante). Sobran 88K. Huecos: 100, 500, 200, 88, 600.

100	500	200	A 212	88	600
-----	-----	-----	-------	----	-----

Paso 2 — Asignar B = 417K. Candidatos: 500 y 600, con sobrantes **83** y 183. Gana el **500**. Sobran 83K. Huecos: 100, 83, 200, 88, 600.

100	B 417	83	200	A 212	88	600
-----	-------	----	-----	-------	----	-----

Paso 3 — Asignar C = 112K. Candidatos: 200 y 600, con sobrantes **88** y 488. Gana el **200**. Sobran 88K. Huecos: 100, 83, 88, 88, 600. Nótese que el 600 sigue *intacto*.

100	B 417	83	C 112	88	A 212	88	600
-----	-------	----	-------	----	-------	----	-----

Paso 4 — Liberar A. El 212K se libera y su vecino derecho *dentro de la misma isla* es el hueco de 88K $\Rightarrow$ **coalescencia**: $212 + 88 = 300$. La isla vuelve a ser un hueco completo de 300K. Huecos: 100, 83, 88, 300, 600.

100	B 417	83	C 112	88	300 (coalescida)	600
-----	-------	----	-------	----	------------------	-----

Paso 5 — Asignar D = 426K ✓. Candidato único: el **600** que mejor ajuste conservó intacto todo este tiempo. Sobrante: $600 - 426 = 174$ K. **La solicitud se atiende sin problema.** Huecos: 100, 83, 88, 300, 174 (total: 745K).

100	B 417	83	C 112	88	300	D 426	174
-----	-------	----	-------	----	-----	-------	-----

Paso 6 — Compactación. B, C y D se mueven a un extremo (955K asignados); hueco único de **745K**. Comprobación: $1700 - (417 + 112 + 426) = 745$ ✓.

B 417	C 112	D 426	hueco único: 745 libres
-------	-------	-------	-------------------------

3. Peor ajuste (Worst Fit)

Regla: elegir siempre el hueco más grande, con la idea de que el sobrante quede grande y “todavía útil”.

Estado inicial. La misma lista de huecos: 100, 500, 200, 300, 600.



Paso 1 — Asignar A = 212K. El hueco mayor es el **600**. Sobrante: $600 - 212 = 388$ K. Huecos: 100, 500, 200, 300, 388. (El hueco más grande ya se gastó.)



Paso 2 — Asignar B = 417K. El mayor ahora es el **500**. Sobrante: $500 - 417 = 83$ K. Huecos: 100, 83, 200, 300, 388. (El segundo hueco grande también se fue.)



Paso 3 — Asignar C = 112K. El mayor ahora es el **388** (dentro de la isla de 600). Sobrante: $388 - 112 = 276$ K. Huecos: 100, 83, 200, 300, 276.



Paso 4 — Liberar A. El 212K se libera, pero su vecino derecho es C (asignado) $\Rightarrow$ *sin coalescencia*. Huecos: 100, 83, 200, 300, 212, 276 (total: **1171K**).



Paso 5 — Asignar D = 426K. El hueco mayor disponible es 300K < 426 K $\Rightarrow$ **FALLA**. Los huecos grandes (600 y 500) se destruyeron en los pasos 1–2, y cuando llegó la solicitud grande ya no había dónde ponerla. El estado no cambia.

FALLA: 1171K libres pero el hueco mayor es de 300K (fragmentación externa)

Paso 6 — Compactación. B y C a un extremo; hueco único de **1171K**. Comprobación: $1700 - (417 + 112) = 1171 \checkmark$.



4. Comparación y conclusión

	Primer ajuste	Mejor ajuste	Peor ajuste
Solicitudes atendidas	3 de 4	4 de 4	3 de 4
D = 426K (paso 5)	falla	se atiende	falla
Huecos antes de compactar	6	5	6
Huevo mayor antes de compactar	300K	300K	300K
Coalescencia al liberar A	no	sí (212+88=300)	no
Memoria libre final	1171K	745K	1171K

Idea clave

El algoritmo que mejor utiliza la memoria en este caso es MEJOR AJUSTE: es el único que atiende las cuatro asignaciones sin necesitar compactación, porque *conserva intacto el hueco de 600K* hasta que llega la solicitud grande de 426K. Peor ajuste hace exactamente lo contrario (consume el 600 y el 500 en los dos primeros pasos) y primer ajuste quema el 600 en el paso 2 por el orden de escaneo. Además, con mejor ajuste la liberación de A sí produce coalescencia, porque A vivía sola en su isla.

Observación típica de examen

- La regla de 10K de fragmentación interna **nunca se activa** (sobrante mínimo: 83K); hay que decirlo explícitamente, no ignorarlo.
- La falla de D en primer y peor ajuste es el ejemplo de libro de **fragmentación externa**: 1171K libres totales, imposible atender 426K por estar dispersos. Como no hay compactación por demanda, la solicitud se rechaza; tras la compactación del paso 6, el hueco único ya podría atenderla.

Método general para resolver cualquier variante

(1) Anota la lista de huecos en orden físico y reléela tras *cada* operación. (2) Para cada asignación aplica la regla del algoritmo sobre la lista *actual* y verifica la regla de fragmentación interna antes de decidir si partes el hueco. (3) Al liberar, revisa *solo dentro de la isla* si hay huecos vecinos para coalescer. (4) Si una solicitud no cabe, declárala fallida (sin compactación por demanda) y deja el estado intacto. (5) Al compactar, suma lo asignado a un extremo y todo lo libre al otro: el hueco final debe cuadrar con total – asignado (usa esto como comprobación).